

Language Models

Objective The goal of this assignment is to build a language model and perform several key analyses. Specifically, you will develop and train a language model using the Brown corpus (`nltk.corpus.brown`) as training set and apply it on Treebank corpus (`nltk.corpus.treebank`) as test set, using the file `language_model.py`; your job is to fill in the missing functions in this file so that it does what it's supposed to do. In addition, you will find sentences from the test dataset based on perplexity and look for interesting patterns. The assignment has several key components:

Data Preparation First, we need to collect word counts so that we have a vocabulary. This is done by the `train_seen` function. Modify this function so that it will keep track of all of the tokens in the training corpus and their counts.

Then, complete the `vocab_lookup` function. This should return a unique identifier for a word, or a common “unknown” identifier for words that do not meet the `unk_cutoff` threshold. You can use strings as your identifier (e.g., leaving inputs unchanged if they pass the threshold) or you can replace strings with integers (this will lead to a more efficient implementation). The unit tests are engineered to accept both options.

After you do this, then the `finalize` and `sensor` functions should work (but you don't need to do anything). But check that the appropriate unit tests are working correctly.

Estimation After you've finalized the vocabulary, then you need to add training data to the model. This is the most important step! Modify the `add_train` function so that given a sentence it keeps track of the necessary counts you'll need for the probability functions later. You will probably want to use default dictionaries or probability distributions. Finally, given the counts that you've stored in `add_train`, you'll need to implement probability estimates for contexts. These are the required probability estimates you'll need to implement:

- `mle`: Simple division of counts for that observation by total counts for the context
- `laplace`: Add one to all counts

Now if you run the main section of the `language_model.py` file, you'll get per-sentence reports of perplexity. Take a look at what sentences are particularly hard or easy (high vs. low perplexity; you don't need to turn anything in here, however).

Exploration Find sentences from the test dataset that get really low vs. high perplexities for each of the estimation schemes (you may want to write some code to do this). Can you find any patterns? Turn in your findings and discussion as `report.txt`.

(OPTIONAL) Extra Credit Extra Credit (make sure they don't screw up required code / functions that will be run by the autograder):

- `dirichlet`: Add a specified parameter greater than zero to all counts
- `jelinek_mercer`: Interpolate between probability distributions with parameter λ
- `kneser_ney`: Use discounting and prefixes with discount parameter δ and concentration parameter α to implement interpolated kneserney.
- Implement a function to produce English-looking output (return an iterator or list) from your language model (function called `sample`)
- Make the code really efficient for reading in sequences of characters

FAQ

- Q: Why are there two passes over the data?
A: The first pass establishes the vocabulary, the second pass accumulates the counts. You could in theory do it in one pass, but it gets much more complicated.
- Q: What if the counts of `<s>` and `</s>` fall below the threshold?
A: They should always be included in the vocabulary.
- Q: And what about words that fall below the threshold?
A: They must also be represented, so the vocab size will be the number of tokens at or above the UNK threshold plus three (one for UNK, one for START, and one for END).
- Q: What happens when I try to take the log of zero?
A: Return `kNEG_INF` instead.
- Q: Do I tune the hyperparameters for interpolation, discount, etc.?
A: No, that's not part of this assignment.

Submission Guideline

Submit a zip file named `NAME_STUDENTID.zip` containing:

1. `language_model.py`: the file with missing components added as described above.
2. `report.txt`: exploratory analysis on sentences with low vs high perplexity and patterns across estimation schemes.
3. (optional) `language_model_extra.py`: supplementary file for extra credit (as indicated above).

Submit your zip file through the link on Blackboard. Submissions elsewhere won't be considered.
